

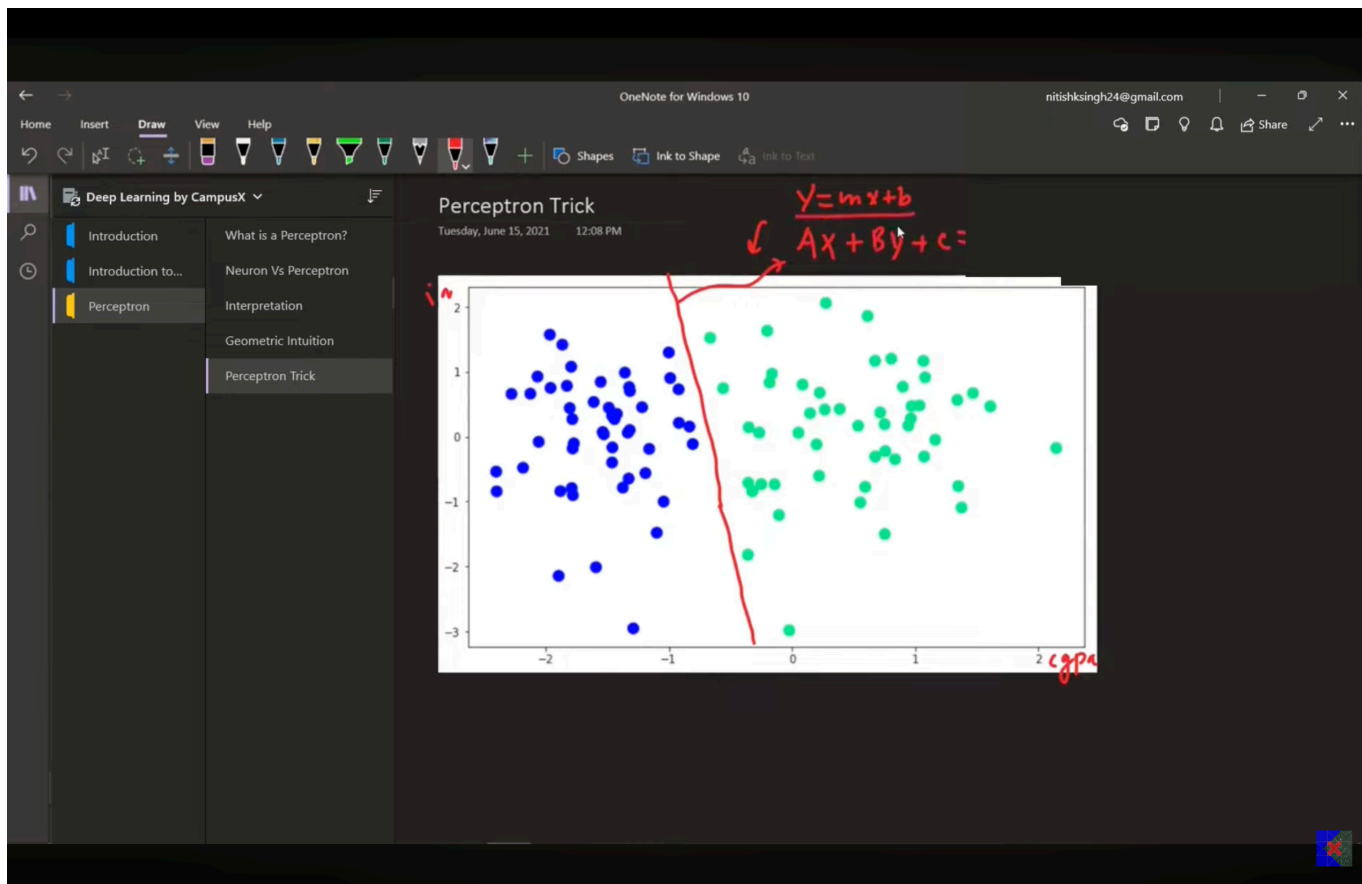
### 3. How to train a Perceptron?

## How to Train a Perceptron

### Problem Setup

Consider training a model based on student data where:

- **Y-axis:** IQ of students
- **X-axis:** CGPA
- **Classes:**
  - Green points = students placed
  - Red points = students not placed



### Linear Decision Boundary

The general equation of a straight line in logistic regression uses the standardized form:

$$ax_1 + bx_2 + c = 0$$

Where:

- $x_1 = \text{CGPA}$
- $x_2 = \text{IQ}$

If we add another parameter (e.g., 10th marks), the equation becomes:

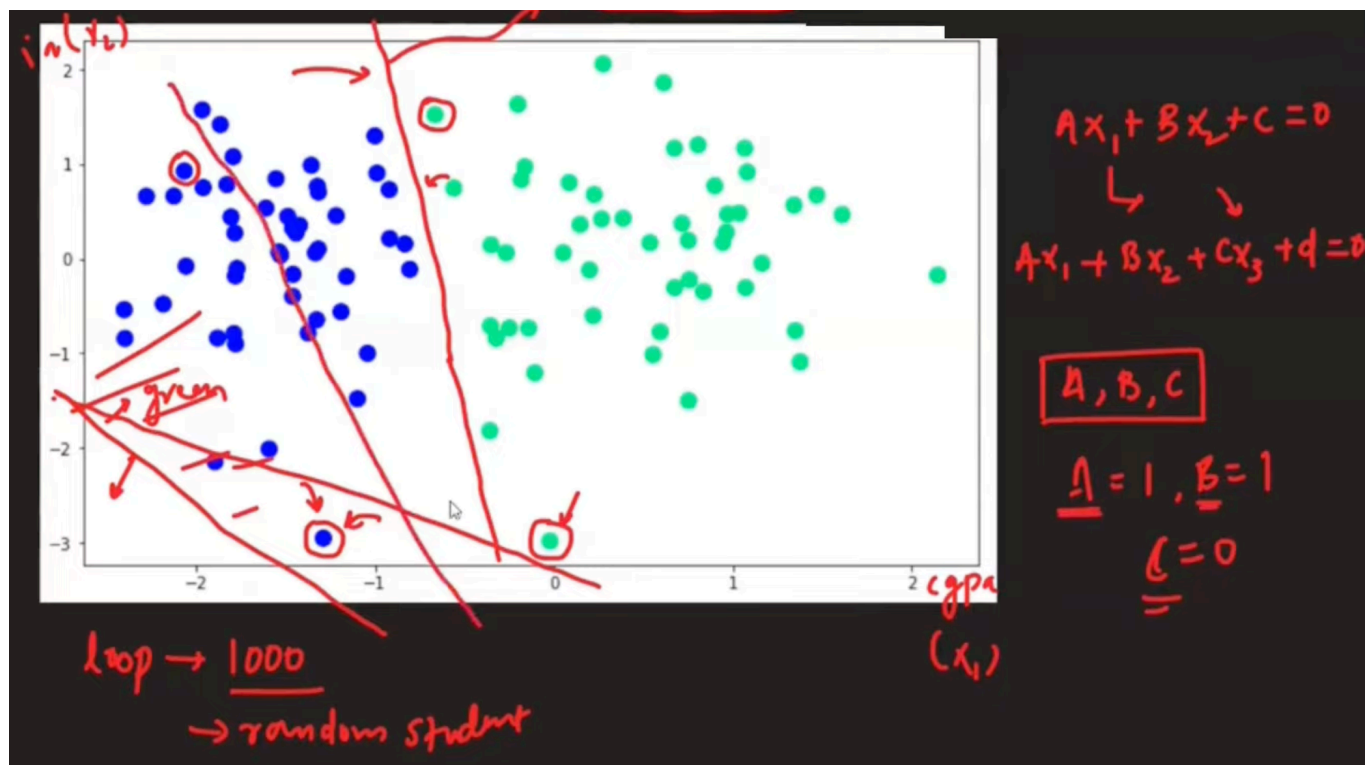
$$ax_1 + bx_2 + cx_3 + d = 0$$

Our goal is to discover values of  $a$ ,  $b$ , and  $c$  that draw a line classifying students as placed or not placed.

## The Perceptron Trick

### Algorithm Steps

1. **Initialize:** Start with random values of  $a$ ,  $b$ ,  $c$ 
  - Example:  $a = 1$ ,  $b = 1$ ,  $c = 0$  (assumes all students will be placed)
2. **Iterate:** Run a loop (typically 1,000 times)
  - For each iteration, select a random student
  - Check if the student is on the correct side of the classification line
3. **Correct Misclassifications:** When a point is on the wrong side of the line, apply a transformation to move the line so the point is correctly classified



### Convergence Criteria

**Option 1:** Run for a fixed number of iterations (1,000 or 10,000)

**Option 2:** Run until convergence

- After each iteration, count misclassified points
- Stop when zero misclassified points remain
- This complete pass through the data is called an **epoch**

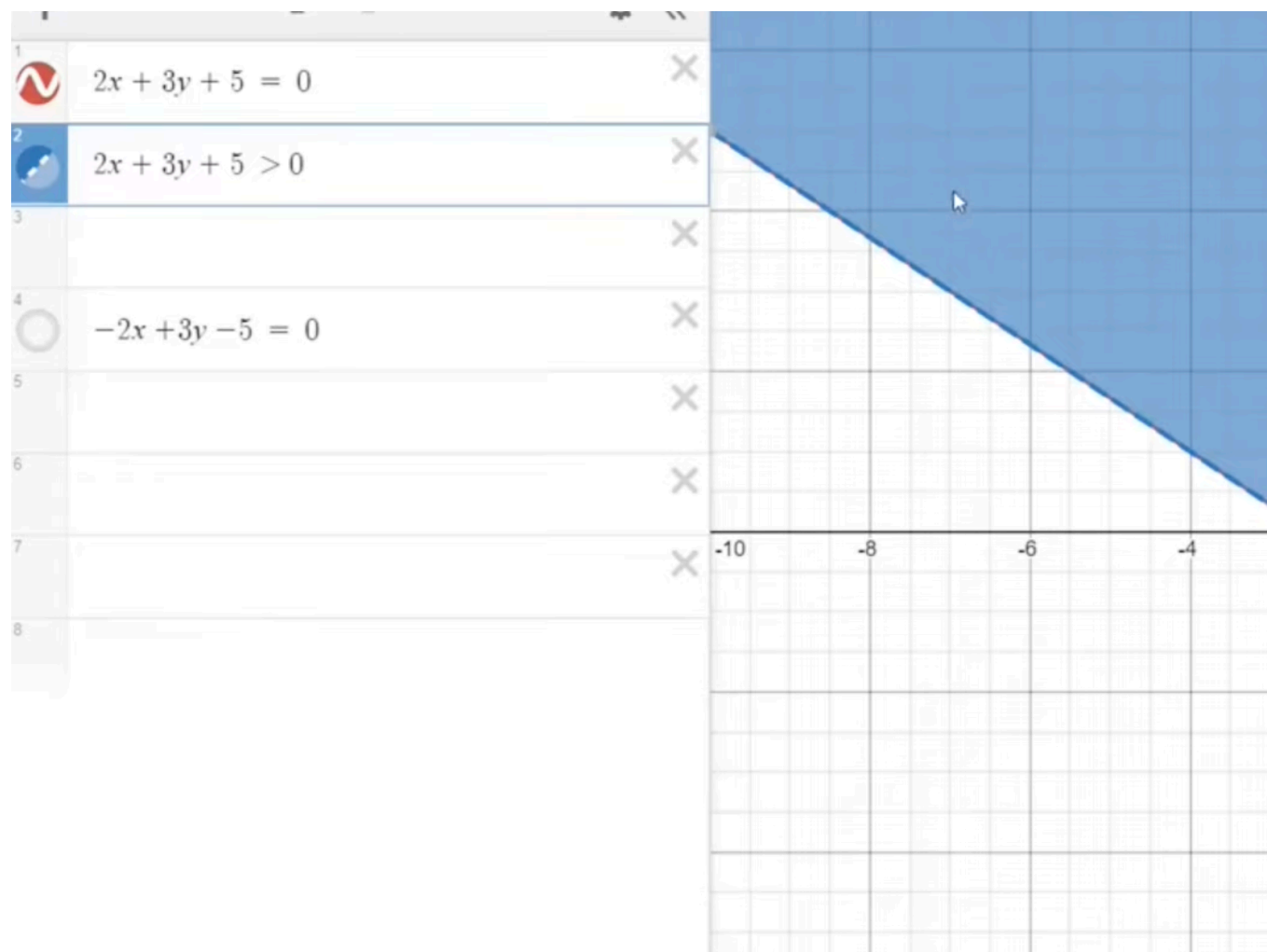
---

## Identifying Positive and Negative Regions

Using an equation with inequality operators, we can visualize regions:

$$2x + 3y + 5 > 0$$

This highlights the region on the graph that is positive for this equation.



---

## Transforming a Line

## General Form

$$Ax + By + C = 0$$

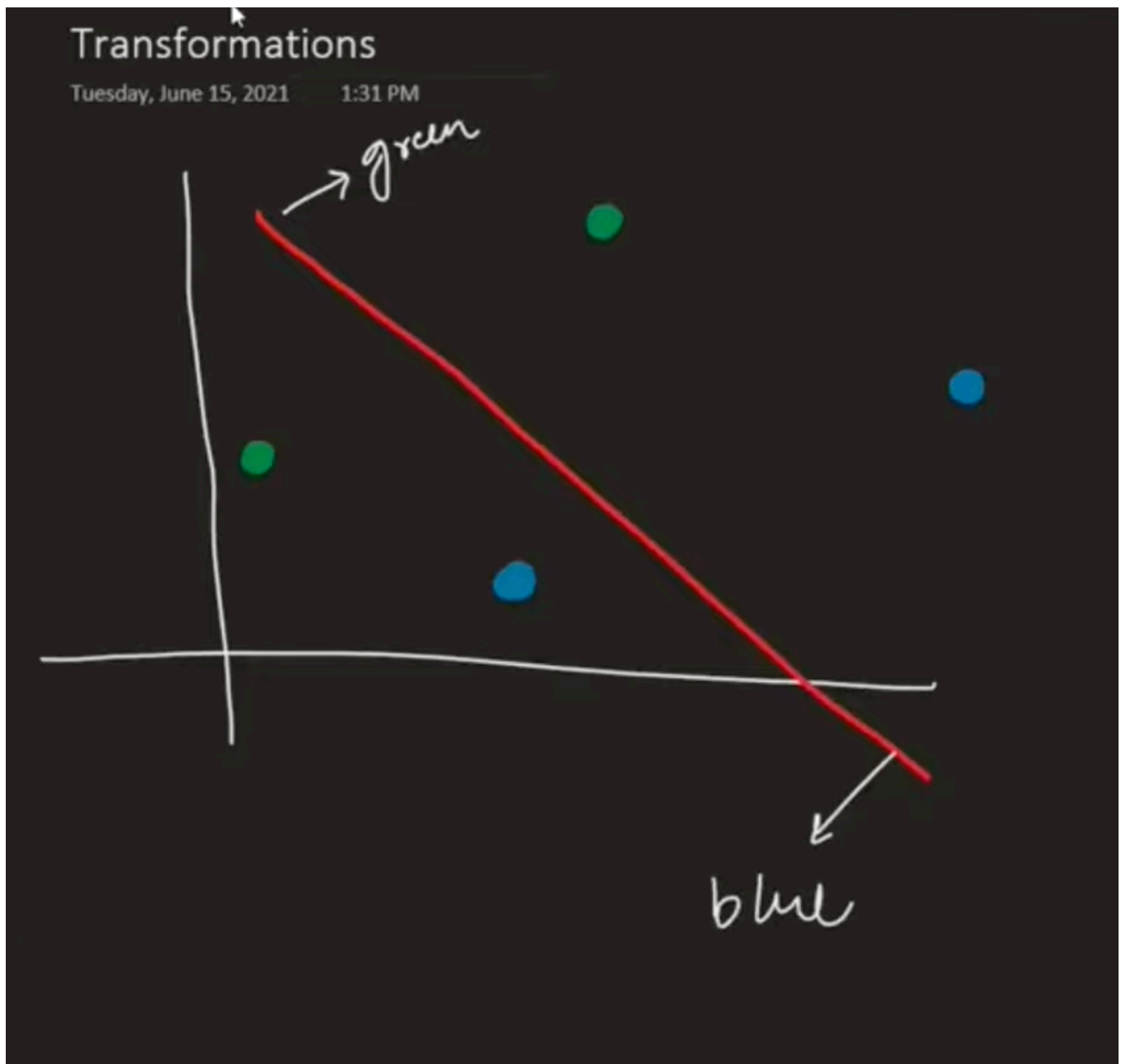
### Effects of changing coefficients:

- Changing  $C$  shifts the line (changes  $x$  and  $y$  intercepts)
- Changing  $A$  rotates the line (affects slope relative to  $x$ -axis)
- Changing  $B$  rotates the line (affects slope relative to  $y$ -axis)

## Example Problem

**Given:** Line with equation  $2x + 3y + 5 = 0$

**Misclassified points:**  $(4, 5)$  and  $(1, 3)$



## Transformation Rules

**Rule 1:** If a **negative point** is in the **positive region**:

- Add 1 to its coordinates:  $(x, y, 1)$
- Subtract from line coefficients to get new equation

**Rule 2:** If a **positive point** is in the **negative region**:

- Add 1 to its coordinates:  $(x, y, 1)$
- Add to line coefficients to get new equation

## Manual Calculation Example

|               |    |    |   |
|---------------|----|----|---|
| Coefficients: | 2  | 3  | 5 |
| Point:        | 4  | 5  | 1 |
| <hr/>         |    |    |   |
| Difference:   | -2 | -2 | 4 |

New equation:  $-2x - 2y + 4 = 0$

---

## Learning Rate

In machine learning, we don't apply all transformations at once. We use a **learning rate** (typically small, like 0.1) to make gradual adjustments.

## Formula

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \text{coordinates}$$

Where  $\alpha$  is the learning rate.

**Example:** Instead of subtracting 4 from 2 directly, we subtract  $0.1 \times 4 = 0.4$  when the learning rate is 0.1.

---

## Perceptron Algorithm

### Notation

Standard form:  $Ax + By + C = 0$

Machine learning notation:  $w_0 + w_1x_1 + w_2x_2 = 0$

Define  $x_0 = 1$ , so:  $w_0x_0 + w_1x_1 + w_2x_2 = 0$

Sigma notation:

$$\sum_{i=0}^2 w_i x_i = 0$$

## Detailed Algorithm

**Parameters:**

- Epochs = 1000
- Learning rate  $\eta = 0.1$

```
for i in range(epochs):  
    Randomly select a point (student)  
  
    if x_i belongs to negative region AND  $\Sigma(w_i \times x_i) \geq 0$ :  
        # Student not placed but model predicts placed  
         $w_{\text{new}} = w_{\text{old}} - \eta \times x$   
  
    if x_i belongs to positive region AND  $\Sigma(w_i \times x_i) \leq 0$ :  
        # Student placed but model predicts not placed  
         $w_{\text{new}} = w_{\text{old}} + \eta \times x$ 
```

Where each variable (except learning rate) is a matrix.

## Simplified Algorithm

The two conditions can be unified into a single update rule:

$$w_n = w_o + \eta(y_i - \hat{y}_i)x_i$$

**Final algorithm:**

```
for i in range(epochs):  
    Select a random student (i)  
    Update coefficients:  $w_{\text{new}} = w_{\text{old}} + \eta(y_i - \hat{y}_i)x_i$ 
```

Where:

- $y_i$  = actual label (0 or 1)
- $\hat{y}_i$  = predicted label

- $x_i$  = feature vector
  - $\eta$  = learning rate
- 

**Note:** The implementation of this algorithm can be found in `perceptron-trick.ipynb`.